Arduino-Based Environmental Monitoring System

Personal Engineering Project — Technical Report

Author:	Emmanuel Phiri
University:	University of the West of England (UWE Bristol)
Degree:	Electrical and Electronics Engineering
Date:	April 2026
GitHub Repository:	https://github.com/pabllo1127/Environmental-Monitoring-System

Table of Contents

1. Executive Summary

This report documents the design, development, and implementation of a personal engineering project: an Arduino-based environmental monitoring system. Driven by a genuine interest in embedded systems and smart home technology, this project was undertaken independently to develop practical engineering skills beyond the classroom.

The system monitors real-time environmental conditions including temperature, humidity, and ambient light levels, providing intelligent alerts to help users maintain ideal living conditions. The project covers the full engineering development cycle: component selection, firmware development in C++, PCB design using KiCad, mechanical enclosure design in SolidWorks, and 3D printing.

The final system integrates a DS3231 real-time clock, DHT11 temperature and humidity sensor, LDR light sensor, 20x4 I2C LCD display, active buzzer alert system, and two user input buttons, all housed within a custom-designed 3D-printed enclosure. All project files are available at: https://github.com/pabllo1127/Environmental-Monitoring-System

2. Introduction and Motivation

The idea for this project came from a personal interest in how embedded systems can be used to improve everyday environments. Maintaining the right temperature, humidity, and lighting in a room has a real impact on comfort, health, and energy usage, yet most people have no easy way to monitor these conditions in real time.

Rather than purchasing an off-the-shelf solution, the goal was to design and build one from scratch, applying skills developed through the Electrical and Electronics Engineering degree at UWE Bristol. This approach provided hands-on experience with embedded firmware, analogue circuit design, PCB layout, and mechanical CAD — skills directly relevant to a career in engineering.

The project was self-directed from concept to completion, including defining requirements, selecting components, writing and debugging firmware, designing the PCB, modelling the enclosure, and documenting the full process in this report.

3. System Requirements and Objectives

3.1 Functional Requirements

ID	Requirement	Priority
FR1	Measure ambient temperature in degrees Celsius	High
FR2	Measure relative humidity as a percentage	High
FR3	Measure ambient light level via LDR	High
FR4	Display real-time clock (hours, minutes, seconds)	High
FR5	Display all sensor readings on LCD	High
FR6	Alert user via buzzer when thresholds are exceeded	High
FR7	Allow user to silence buzzer via button	Medium
FR8	Allow user to toggle between display screens	Medium
FR9	Provide contextual advice (e.g. open window, turn off light)	Medium

3.2 Non-Functional Requirements

The system must operate reliably from a 9V battery supply, consume minimal power during idle operation, and fit within a compact enclosure suitable for wall mounting. The firmware must be maintainable and well-commented. The PCB must conform to standard manufacturing tolerances and the enclosure must be 3D printable using PLA material.

3.3 System Thresholds

Parameter	Min Threshold	Max Threshold	Action if Exceeded
Temperature	16 degrees C	25 degrees C	Alert: turn on/off heating
Humidity	None	60%	Alert: open a window
Light (daytime 6am-10pm)	600 (ADC units)	None	Alert: turn on lights
Light (night-time 10pm-6am)	None	700 (ADC units)	Alert: turn off lights

4. Hardware Design and Component Selection

The hardware architecture centres on the Arduino Uno R3 microcontroller, chosen for its ease of programming, broad community support, and sufficient I/O for the required peripherals. Components were selected based on availability, cost, and suitability for the application.

Component	Model	Interface	Purpose
Microcontroller	Arduino Uno R3	None	Main processing unit
Display	LCD 20x4 + PCF8574 I2C backpack	I2C (A4/A5)	Display data and alerts
Temp/Humidity Sensor	DHT11	Digital (Pin 7)	Measure temperature and humidity
Light Sensor	LDR + 100k resistor	Analogue (A0)	Measure ambient light level
Real-Time Clock	DS3231	I2C (A4/A5)	Accurate timekeeping
Buzzer	Active buzzer	Digital (Pin 4)	Audible alert system
Button 1	Tactile push button	Digital (Pin 5)	Silence buzzer
Button 2	Tactile push button	Digital (Pin 6)	Toggle LCD screen
Power Supply	9V battery pack	Barrel jack	Standalone power

4.1 Sensor Selection Rationale

The DHT11 was selected as the temperature and humidity sensor due to its simple single-wire digital interface, low cost, and sufficient accuracy for this application (plus or minus 2 degrees C, plus or minus 5% RH). The DS3231 real-time clock was chosen for its superior accuracy and I2C interface, which shares the existing I2C bus with the LCD, keeping the design clean and minimising pin usage. The LDR is configured in a voltage divider with a 100k resistor, producing an analogue voltage proportional to ambient light, read by the Arduino ADC on pin A0.

4.2 Pin Allocation

Arduino Pin	Connected To	Type
A0	LDR voltage divider midpoint	Analogue Input
A4 (SDA)	LCD I2C SDA + DS3231 SDA	I2C Data
A5 (SCL)	LCD I2C SCL + DS3231 SCL	I2C Clock
Pin 4	Active buzzer (+)	Digital Output
Pin 5	Silence button	Digital Input

Pin 6	Toggle button	Digital Input
Pin 7	DHT11 data	Digital Input
5V	LCD, DS3231, DHT11, Buttons	Power
GND	All component grounds	Ground

5. Circuit Design and PCB Layout

5.1 Schematic Design

The circuit schematic was designed using KiCad 9.0. The I2C bus (SDA/SCL) is shared between the LCD module (address 0x27) and the DS3231 RTC (address 0x68), demonstrating multi-device I2C communication where each device responds only to its unique address. The LDR is wired in a pull-down voltage divider configuration. Both push buttons connect between the Arduino digital pin and 5V supply. The full schematic is available in the GitHub repository.

5.2 PCB Layout

The PCB was designed in the KiCad PCB editor following schematic completion and ERC verification. The board measures approximately 170mm x 130mm. The LCD is positioned at the top for visibility, the Arduino centrally as the main hub, and smaller components arranged around it to minimise trace lengths. Traces are routed on F.Cu and B.Cu layers. The full PCB layout file is available in the GitHub repository at https://github.com/pabllo1127/Environmental-Monitoring-System

6. Firmware Development

6.1 Software Architecture

The firmware was written in C++ using the Arduino IDE. It uses non-blocking timing with millis() to keep the main loop responsive for button handling and LCD updates. The code is structured into clear sections: clock management, sensor reading, threshold evaluation, button handling, buzzer control, and LCD display management. The full firmware is available in the GitHub repository.

6.2 Display Logic

The 20x4 LCD displays two screens cycling every 5 seconds or on button press. Screen 0 shows time, temperature, humidity, and light status across all four lines. Screen 1 shows a conditions summary with advisory messages. When any alert triggers, the display jumps immediately to Screen 1 to notify the user.

6.3 Alert Logic

The core threshold evaluation and alert code:

```
bool tooDark = (hours >= 6 && hours < 22 && lightval < 600); bool tooBright = ((hours >= 22 || hours < 6) && lightval > 700); bool tooHot = (temp > 25); bool tooCold = (temp < 16); bool tooHumid = (humidity > 60); bool alertNeeded = tooDark || tooBright || tooHot || tooCold || tooHumid; if (alertNeeded && !buzzerSilenced) { digitalWrite(buzzerPin, HIGH); } else { digitalWrite(buzzerPin, LOW); }
```

6.4 Libraries Used

Library	Author	Purpose
Wire.h	Arduino	I2C communication
LiquidCrystal_I2C.h	Frank de Brabander	I2C LCD control
DHT11.h	Dhruba Saha	DHT11 sensor reading
RTClib.h	Adafruit	DS3231 RTC interface

7. Enclosure Design and 3D Printing

7.1 Design Concept

The enclosure was designed in SolidWorks as a flat, wall-mountable housing inspired by domestic devices such as thermostats. The design keeps the LCD clearly visible while maintaining a clean, unobtrusive appearance. It was modelled as a solid block and hollowed using the Shell feature with a 3mm wall thickness. The SolidWorks file is available in the GitHub repository.

7.2 Cutouts and Features

Feature	Location	Purpose
LCD window	Front face, upper centre	LCD visibility
Button hole 1	Front face, left of LCD	Silence button access
Button hole 2	Front face, right of LCD	Toggle button access
DHT11 aperture	Side face	Sensor ventilation and access
LDR aperture	Side face	Ambient light sensing
Battery connector hole	Bottom face	9V power input

7.3 3D Printing Specifications

Material: PLA. Layer height: 0.2mm. Infill: 20%. No supports required. The 3mm wall thickness provides good structural integrity without excessive print time.

8. System Testing and Results

8.1 Component-Level Testing

Component	Test Method	Result
LCD	Hello World sketch via I2C	Pass - after library and address verification
DHT11	Serial output of temp/humidity	Pass - 23 degrees C, 55% RH typical
LDR	Serial output of ADC value	Pass - 53 (dark) to 1014 (torch)
DS3231 RTC	Serial output of time	Pass - accurate timekeeping confirmed
Buzzer	digitalWrite HIGH/LOW test	Pass - audible alert confirmed
Button 1	digitalRead with Serial output	Pass - debounced correctly
Button 2	digitalRead with Serial output	Pass - debounced correctly

8.2 Full System Testing

All components were tested together as a complete system. Alert conditions were triggered manually by covering the LDR, applying heat to the DHT11, and breathing on the humidity sensor. All conditions correctly triggered the buzzer and switched the LCD to the alert screen. Both buttons functioned reliably with debouncing. The DS3231 maintained accurate time across power cycles.

9. Challenges and Solutions

9.1 LCD Not Displaying Text

The LCD showed only black blocks. After systematic debugging, the issue was identified as using the wrong library. Switching to LiquidCrystal_I2C by Frank de Brabander and verifying the I2C address as 0x27 resolved it immediately.

9.2 DHT11 Sensor Damage

Two DHT11 sensors were damaged due to incorrect pin orientation during initial testing. The issue was resolved by carefully verifying the module pin labelling before connection, and selecting the correct DHT11 library by Dhruba Saha.

9.3 RTC Module Failure

The initially used DS1302 RTC consistently read 0:0:0. After exhausting all wiring combinations it was concluded the module was faulty. Replacing it with the DS3231 resolved the issue and provided better accuracy.

9.4 Button Debouncing

Buttons showed rapid state bouncing on press. Solved by implementing state-change detection tracking the previous button state with a 50ms debounce delay.

9.5 KiCad LCD Footprint

The LCD 20x4B footprint was not in the standard KiCad libraries. It was sourced from SnapEDA and manually imported into the footprint library manager.

10. Conclusion and Future Work

10.1 Conclusion

This personal project successfully produced a fully working environmental monitoring system built entirely from scratch. It meets all the original objectives: real-time monitoring of temperature, humidity, and light; clear LCD display with multiple screens; and an intelligent buzzer alert system with user controls.

The project was a valuable exercise in applying theoretical engineering knowledge to a practical problem. It developed skills in embedded firmware, circuit design, PCB layout, and mechanical CAD — all within a self-directed, independent context. All deliverables are available at https://github.com/pabllo1127/Environmental-Monitoring-System

10.2 Future Work

- Bluetooth connectivity for data logging to a smartphone

- Integration with smart home platforms such as Home Assistant via MQTT

- CO2 or air quality sensor for more comprehensive monitoring

- Relay output to directly control heating or ventilation

- Low-power sleep mode to extend battery life

- Wi-Fi via ESP32 for a remote web dashboard

11. References

[1] Arduino. (2024). Arduino Uno R3 Documentation. https://docs.arduino.cc/hardware/uno-rev3

[2] Saha, D. (2023). DHT11 Arduino Library. https://github.com/dhrubasaha08/DHT11

[3] Adafruit Industries. (2024). RTClib Library Documentation. https://github.com/adafruit/RTClib

[4] de Brabander, F. (2023). LiquidCrystal_I2C Library. https://github.com/fdebrabander/Arduino-LiquidCrystal-I2C-library

[5] KiCad EDA. (2024). KiCad 9.0 Documentation. https://docs.kicad.org

[6] SnapEDA. (2024). LCD-20X4B Component Symbol and Footprint. https://www.snapeda.com

[7] Dassault Systemes. (2024). SolidWorks 3D CAD Documentation. https://www.solidworks.com

[8] Maxim Integrated. (2015). DS3231 Datasheet. https://datasheets.maximintegrated.com/en/ds/DS3231.pdf

[9] Aosong Electronics. (2010). DHT11 Humidity and Temperature Sensor Datasheet.

[10] Emmanuel Phiri. (2026). Environmental Monitoring System - GitHub. https://github.com/pabllo1127/Environmental-Monitoring-System